

Снемане на характеристики и архитектурно планиране на софтуерна система

Цел на упражнението

Целта на това упражнение е да се научим да анализираме изискванията за софтуерна система, да систематизираме основните ѝ характеристики и да изберем подходяща архитектура за нейното създаване. Вместо директна реализация на код, акцентът пада върху процеса на планиране, анализ на изисквания и проектиране на структурата на системата.

Описание на казуса

Фирма за организиране на събития поръчва нова информационна система, която да подпомага организирането на големи и малки събития от всякакъв тип. Системата трябва да предоставя функционалности за управление на събития, зали и местоположения, както и за резервации на билети, управление на потребителски данни и роли. Клиентът предоставя основни изисквания и очаква от разработчиците да извлекат конкретни детайли, да анализират задачите и да създадат план за реализиране на архитектурата.

Изисквания от клиента

1. Функции на системата:

- **Управление на събития:**
 - Организаторите във фирмата могат да създават събития с основна информация като:
 - Заглавие.
 - Описание.
 - Дата и час.
 - Цена на билета.
 - Максимален капацитет на местата.
- **Управление на зали и местоположения:**
 - Каталогизиране на залите с данни като:
 - Име на залата, локация и координати.
 - Максимален капацитет (брой места).
 - Разпределение на местата (чрез визуална 2D карта).
 - Проверка за конфликти (дали залата не е резервирана за друго събитие по същото време).
- **Управление на резервации:**

- Посетителите могат:
 - Да разглеждат наличните събития и зали.
 - Да избират конкретни места и да правят резервации.
 - Да плащат за билети онлайн.
- Системата трябва да обработва резервации в реално време с механизъм за блокиране на места, за да се избегне двойно резервиране.
- **Потребителски роли:**
 - Организатори:
 - Създаване и управление на събития.
 - Управление на текущи резервации.
 - Преглеждане на отчети за продадени билети.
 - Посетители:
 - Разглеждане на събития.
 - Резервиране и запазване на билети.
 - Получаване на известия.
 - Администратори:
 - Наблюдение и управление на системата.
 - Управление на зали и местоположения.
 - Генериране на отчети за събитията.

2. Допълнителни функции за администрация:

- Генериране на отчети за:
 - Продадените билети и приходите по събития.
 - Наличността на местата в залите.
- Мониторинг на заетостта на залите и тяхното планиране.
- Обновяване на информация за капацитет и достъпност на зали.

3. Известия:

- Отпращане на нотификации:
 - Потвърждаване на резервации по e-mail или SMS.
 - Напомняния за приближаващи събития (например, 24 часа преди събитието).
- Генериране на отчети за събития, които не са достигнали минимален капацитет.

4. Ограничения и бъдещи разширения:

- Системата трябва да може да обработва мащабни събития с до 10,000 резервации.
- Общуването с чувствителни данни (например плащания) трябва да бъде защитено.
- Поддръжка на бъдещи интеграции, например:
 - Приложение за мобилни устройства.
 - Интеграция със стрийминг платформи.
 - Програмно управление на клубни карти (с отстъпки за редовни клиенти).

Стъпки за анализ на системата

Анализ на изискванията

Една успешна архитектура се базира на дълбоко разбиране на нуждите на клиента и предоставеното задание. Започнете с внимателно четене на заданието и следвайте тези подетапи:

Съберете функционалните изисквания

- Определете **основните функции** (например управление на събития, резервации);
- Определете пропуски в заданието на клиента, които не са описани, но задължително трябва да се планират;
- Идентифицирайте второстепенните функции или бъдещи разширения (например мобилно приложение или стрийминг интеграция);

Примерно разглеждане на заданието:

- Създаване на събития (заглавие, описание, дата, цена, капацитет).
- Управление на налични зали (максимален капацитет, карта на местата).
- Резервация и плащане в реално време.

Определете нефункционалните изисквания:

Нефункционалните изисквания определят ограниченията на системата:

- Масшабируемост - справяне с натоварвания при големи събития;
- Сигурност - съхранение на лични данни и обработка на плащания, съобразено с GDPR.
- Изпълнение - бърза обработка на заявки, за да се избегнат забавяния при резервации;
- Надеждност - гаранция, че системата работи без проблеми независимо от натоварването;
- Гъвкавост - лесна поддръжка и добавяне на нови функции в бъдеще (например интеграция с мобилно приложение).

Съберете допълнителните въпроси към клиента:

Анализирайте дали има пропуски в заданието и задайте уточняващи въпроси:

- Колко събития и посетители се очаква да се обработват месечно?
- Има ли предпочитания за интеграция с външни системи (например платежни услуги)?
- Какъв е планът за бъдещи разширения (например мобилни приложения)?
- Какви са очакваните времена за реакция при резервации в реално време?

Разделяне на приложението на компоненти

Анализирайте какви са основните части на системата и групирайте функционалностите в компоненти. За всяка част определете какви действия трябва да изпълнява и как ще комуникира с другите компоненти.

Основни компоненти за тази система:

1. Управление на събития:

- Създаване, промяна и премахване на събития.
- Информация за събитията: име, дата, капацитет, цени.
- Връзка със залите (дали залата е свободна на дадена дата).

2. Управление на зали:

- Каталог с налични зали и капацитет.
- Разпределение на местата (2D карта).
- Планиране на зали и избягване на конфликти (двойно резервиране).

3. Управление на резервации:

- Поддържане на реално време при резервиране на билети за събития.
- Проверка за наличност на свободни места.
- Опции за плащане и генериране на потвърждения.

4. Потребителски роли и разрешения:

- Организатори: управление на събития.
- Посетители: резервации, получаване на известия.
- Администратори: наблюдение и администрация.

5. Управление на известия:

- Система за изпращане на email/SMS за потвърждение и напомняния за събития.

6. Административен модул:

- Генериране на отчети за продадени билети и приходи.
- Анализ на заетостта на зали.

3. Оценка на архитектурните стилове

Изборът на архитектура зависи от изискванията за гъвкавост, мащабируемост и надеждност.

Можете да разгледате следните архитектурни подходи:

а) Монолитна архитектура

- Подходяща за малки системи с ограничени функции.
- **Предимства:**
 - По-лесна реализация и управление.
 - Подходяща за MVP или системи с малки натоварвания.
- **Недостатъци:**
 - Ограничена мащабируемост.
 - Трудна поддръжка при добавяне на нови функции.

b) Триредова архитектура

- Разделя системата на три основни части:
 - **Frontend:** Потребителски интерфейс.
 - **Backend:** Логика за управление на събития, резервации и зали.
 - **База данни:** Съхранение на информация за събития, резервации, зали.
- **Предимства:**
 - Лесно разделение на компонентите и задачи.
 - Гъвкавост за бъдещо развитие и интеграции.
- **Недостатъци:**
 - Средна сложност при настройване и поддръжка.

c) Микросървисна архитектура

- Всеки компонент (зали, резервации, събития) работи като отделна микросървисна единица.
- **Предимства:**
 - Висока мащабируемост – всеки компонент може да се разширява независимо.
 - Възможност за екипна работа върху различни части на системата.
 - Подходяща за големи системи с натоварвания.
- **Недостатъци:**
 - По-голяма сложност при релации между микросървиси.
 - Изисква строго планиране и експертност.

d) Хибридна архитектура

- Комбинация от монолит за базови функции и микросървиси за специфични задачи (например реално време за резервации).
- **Предимства:** Гъвкавост и логично надграждане.
- **Недостатъци:** Повишена сложност в сравнение с монолитния вариант.

4. Вземане на решение за архитектура

След сравнение на архитектурните стилове, изберете най-подходящия подход на база анализа. Обосновете избора си, като разгледате въпросите:

- Колко често се очакват промени в изискванията?
- Как ще се справите с натоварване (масови резервации)?
- Колко бързо трябва системата да обработва заявки (реално време)?
- Как ще се обработват бъдещи разширения?

5. Валидиране на заключенията

След създаване на първично архитектурно предложение, трябва да преминете през процес на валидиране:

- Прегледайте компонентните зависимости и дали избраният подход покрива възможността за разширения.
- Симулирайте сценарии за големи натоварвания.
- Проверете как отделните роли (организатор, посетител, администратор) ще взаимодействат със системата.

План 1: Реализация на приложението с монолитна архитектура

Монолитната архитектура комбинира всички функционалности в единна, цялостна система. Подходяща е за по-малки екипи и проекти, които изискват минимална сложност в комуникацията между компонентите.

1. Характеристики на монолитната архитектура

- Всички модули (управление на зали, събития, резервации, плащания и известия) се създават и поддържат в една кодова база.
- Един бекенд сървър се грижи за цялата логика.
- Всички функционалности използват споделена база данни.

2. Основни стъпки за реализация

Етап 1: Събиране на изисквания

- Съставяне на пълния списък с функционалности (управление на зали, събития, резервации и потребители).
- Детайлно планиране на структурата на базата данни.

Етап 2: Дизайн на архитектурата

- Определяне на слоевете на приложението:
 - **Потребителски интерфейс (Frontend):**
 - Реализира се като SPA (Single Page Application) с помощта на библиотека като React, Angular или Vue.js.
 - **Сървърен слой (Backend):** Централизирана REST API услуга, базирана на Node.js или PHP (например Laravel).
 - **База данни:** Например MySQL или PostgreSQL за управление на всички релационни данни.

Етап 3: Реализация на функциите (по компоненти)

1. **Модул за управление на зали:**
 - API за добавяне на зали, тяхното редактиране, капацитет и местоположения.
 - Логика за проверка на конфликти при резервации в дадена зала.
2. **Модул за управление на събития:**
 - API за създаване, редактиране и изтриване на събития.

- Логика за свързване на събитие със съответна зала и за капацитетното ограничение.
- 3. **Модул за управление на резервации:**
 - API за създаване и потвърждение на резервации.
 - Блокиране на свободни места в реално време.
- 4. **Административен модул (роля на администратори):**
 - Генериране на отчети за събития и резервации.
- 5. **Модул за известия:**
 - Изпращане на имейли с потвърждения и напомняния за събития.

Етап 4: Тестване

- Пълно тестване на всички функционалности от край до край.
- Осигуряване на правилна работа на API-то при високо натоварване.

Етап 5: Деплоймент

- Хостване на приложението върху централен сървър (напр. AWS EC2, DigitalOcean).
- Използване на NGINX или Apache за управление на трафика към сървъра.

Предимства на монолитната архитектура

- Бърза разработка на MVP (минимално жизнеспособен продукт).
- По-лесна интеграция и конфигурация на компонентите.
- Подходящо за малки проекти и екипи.

Недостатъци

- Трудно мащабиране – ако даден компонент натоварва цялата система, усложненията засягат останалите части.
- Ограничени възможности за независими актуализации на компонентите.

План 2: Реализация на приложението с микросървисна архитектура

Микросървисната архитектура разделя функционалностите на самостоятелни модули (наречени микросървиси), които взаимодействат чрез API-та.

1. Характеристики на микросървисната архитектура

- Всеки микросървис е отговорен за отделно ниво на функционалност (например управление на зали, резервации, събития и др.).
- Модулите са независими от технологична гледна точка и дори могат да са написани на различни езици (например Node.js за управление на резервации и Python за отчетите).
- Данните се съхраняват в независими бази данни за всеки микросървис.

2. Основни стъпки за реализация

Етап 1: Събиране на изисквания

- Анализиране и разделяне на функциите на самостоятелни домейни (управление на зали, събития, резервации и известия).

Етап 2: Дизайн на архитектурата

- Определяне на микросървисите:
 - 1. Микросървис за управление на зали:**
 - Отговорен за добавяне, редактиране и четене на информация за залите.
 - Работи с отделна база данни за залите.
 - 2. Микросървис за управление на събития:**
 - Управлява CRUD операциите (създаване, четене, актуализиране, изтриване) за събития.
 - Съвързва се с микросървиса за зали, за да проверява наличност.
 - 3. Микросървис за управление на резервации:**
 - Отговорен за логиката за резервиране на места.
 - Поддържа блокирания на места в реално време.
 - 4. Микросървис за известия:**
 - Използва отделна услуга (например AWS SES или Twilio) за изпращане на SMS и e-mail известия.
 - 5. Административен микросървис:**
 - Генерира отчети от данните за резервации и събития.

Етап 3: Комуникация между микросървиси

- Всеки микросървис предоставя REST API или gRPC за комуникация помежду си.
- Може да се използва опашка за съобщения (RabbitMQ, Apache Kafka или Redis), за да се улесни обменът на данни.

Етап 4: Технологичен стек

- **Frontend:** Реализиран с помощта на SPA framework (React, Angular или Vue.js).
- **Микросървиси:**
 - Node.js или Python за REST API.
 - PostgreSQL или MongoDB за независима база данни.
- **Оркестрация:** Kubernetes или Docker Compose за управление на микросървисите.

Етап 5: Деплоймент

- Всеки микросървис е независим и се хоства отделно:
 - Използване на Docker контейнери за всеки микросървис.
 - Хостване на различни облачни платформи (например AWS, Google Cloud).
- Имплементация на load balancer (например AWS Elastic Load Balancer) за управление на трафика.

Етап 6: Тестване и мониторинг

- Тестване на функционалността на всеки микросървис поотделно (unit и integration testing).
- Интеграция на мониторинг инструмент като Grafana или Prometheus.

Предимства на микросървисната архитектура

- Лесно мащабиране на конкретни микросървиси (например, модулът за резервации при високо натоварване).
- Възможност за независимо обновление и надграждане на отделните услуги.
- Подходящо за мащабни системи с много компоненти.

Недостатъци

- Повишена сложност при първоначалната реализация.
- Усилена комуникация между микросървиси.
- Изисква опитен екип за управление на Docker/Kubernetes.

Заклучение

- **Монолитна архитектура:** Препоръчва се за малки до средни по мащаб системи, или за първоначалната версия на продукта (MVP).
- **Микросървисна архитектура:** Подходяща за по-големи проекти, които изискват висока мащабируемост, надеждност и бъдещи разширения.